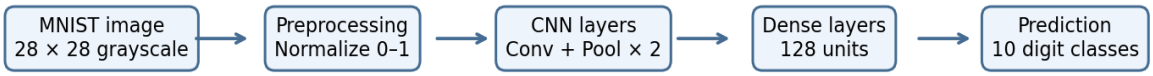
MINOR PROJECT REPORT

CNN Handwritten Digit Recognition

Image Classification using Convolutional Neural Network
with a Streamlit Web Application



Student Name	Gautami Balaso Kamble
Institute	Tatyasaheb Kore Institute of Engineering and Technology (TKIET)
Selected Dataset	MNIST handwritten digits
Application Topic	Handwritten digit prediction
Project Type	Individual minor project
Date	November 2026

This report describes a CNN-based classifier trained on MNIST and a Streamlit interface designed to accept an image and display a predicted digit.

1. Objective and Dataset

Objective. The goal of this project is to build a Convolutional Neural Network (CNN) that recognizes handwritten digits from 0 to 9, evaluate its performance on unseen test images, and provide a simple web interface for trying an image prediction.

Problem statement. Handwritten digits can vary in shape, size, stroke thickness, and writing style. A CNN learns visual patterns from labelled examples and uses those patterns to classify a digit.

Dataset description

The project uses the MNIST dataset, a standard dataset of grayscale images of handwritten digits. Each image is 28 × 28 pixels and belongs to one of ten classes: 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9.

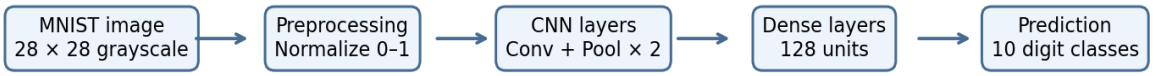
Property	Details
Training set	60,000 labelled images
Test set	10,000 labelled images
Image dimensions	28 × 28 pixels
Image type	Grayscale
Number of classes	10 digits (0–9)
Label format	Integer class label

Preprocessing

1. Load MNIST through TensorFlow/Keras. 2. Convert pixel values to floating-point values. 3. Divide pixel values by 255 so the range changes from 0–255 to 0–1. 4. Reshape each image to 28 × 28 × 1 to include the single grayscale channel. 5. Use a validation split during training and reserve the test set for final evaluation.

Why preprocessing matters: consistent input shape and normalized pixel values help the network learn more reliably.

2. CNN Design and Training Workflow



CNN model architecture

Layer	Purpose
Input: 28 × 28 × 1	Receives a grayscale digit image.
Convolution: 32 filters, 3 × 3, ReLU	Learns low-level features such as edges and strokes.
Max pooling: 2 × 2	Reduces feature-map size while retaining useful features.
Convolution: 64 filters, 3 × 3, ReLU	Learns more detailed digit patterns.
Max pooling: 2 × 2	Further reduces spatial dimensions.
Flatten	Converts feature maps into a one-dimensional vector.
Dense: 128 units, ReLU	Combines learned features for classification.
Dense: 10 units, Softmax	Outputs a score/probability for each digit class.

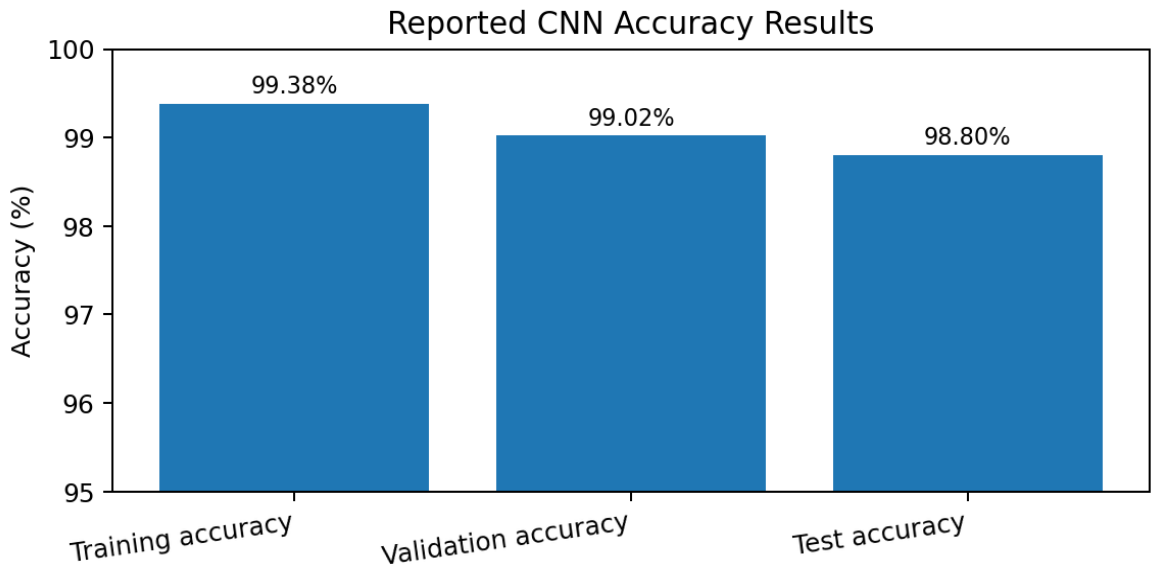
Training configuration

The model is compiled with the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as a metric. It is trained for 5 epochs with a batch size of 64 and a 10% validation split from the training data.

Training workflow: load data → preprocess → create CNN → compile → train → validate → evaluate on test data → predict sample images.

3. Results and Observations

The following values were reported from the project run. Accuracy is the proportion of images classified correctly. Test loss measures the model's prediction error on the held-out test set.



Metric	Reported result
Final training accuracy	99.38%
Final validation accuracy	99.02%
Test accuracy	98.80%
Test loss	0.0386

Interpretation. The reported results indicate that the CNN learned the MNIST digit patterns effectively and achieved high accuracy on unseen test images. The test score is slightly lower than the training score, which is normal because test images were not used to fit the model.

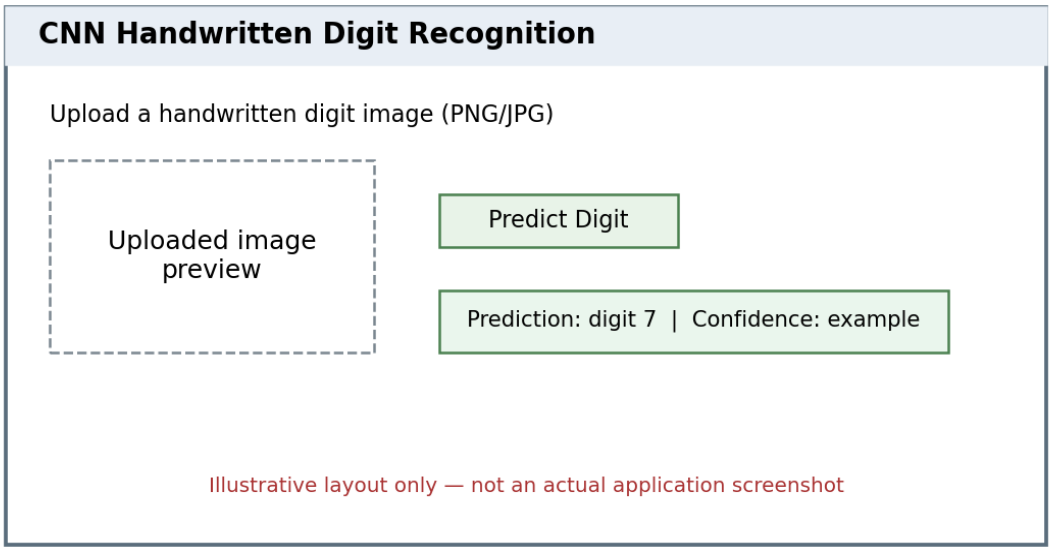
Prediction evidence. The code should display sample test images with both actual and predicted labels. Correct predictions and any incorrect predictions should be captured from the actual run; they are not invented in this report.

4. Streamlit Application

The application is designed to let a user upload an image of a handwritten digit and request a prediction. The CNN returns the most likely digit from 0 to 9 and a confidence percentage derived from the highest softmax output.

Main features

• Clear application title and short instructions.
• Image upload for PNG, JPG, or JPEG files.
• Preview of the uploaded image.
• A button to start prediction.
• Predicted digit, estimated confidence, and a progress indicator.



The figure above is an illustrative layout, not a screenshot of the running app. Replace it with an actual screenshot from the working Streamlit application before final submission.

5. Challenges and Possible Improvements

Challenges: uploaded images may differ from MNIST images in background colour, alignment, stroke thickness, or scale. Resizing an arbitrary photo directly to 28 × 28 pixels can distort the digit. Also, Streamlit reruns a script after user interactions, so model training should be cached or separated from repeated UI actions.

Possible improvements: improve image preprocessing (crop the digit, center it, and handle background inversion); save and reload a trained model to avoid retraining; show a confusion matrix and more sample predictions; and test the interface with a range of handwriting styles.

Conclusion

This project demonstrates an end-to-end image-classification workflow using MNIST, preprocessing, a CNN model, training and test evaluation, and a Streamlit user interface for interactive prediction. The reported test accuracy is 98.80%. The project also highlights the importance of matching uploaded-image preprocessing to the format used during training.

References

1. MNIST handwritten digit database, accessed through TensorFlow/Keras.
2. TensorFlow/Keras documentation for CNN layers and model training.
3. Streamlit documentation for file upload and user-interface components.

FINAL CHECK BEFORE SUBMISSION: add actual screenshots of (1) model.summary() or terminal model summary, (2) training/validation accuracy and loss graphs, (3) actual-versus-predicted sample digits including at least one incorrect prediction if available, and (4) the running Streamlit app. Do not label an illustrative image as a real screenshot.